The Clean Code & Automation Cycle

Made by Javier Gallego and Diego José García

MEDAC
Instituto Oficial de Formación Profesional

LARGE CLASS SMELL

```
public void incrementarRecompensa(long cantidad) {  no usages  Javier Gallego
    Scanner teclado =new Scanner (System.in);
    System.out.println("Introduce la nueva recompensa:");
    cantidad=teclado.nextLong();
    recompensa=cantidad;
}
```

Before

We extracted the method of increasing rewards into a new class that will handle the function of returning the bounty to characters and put it inside the old one.

```
public void incrementarRecompensa(long cantidad) {  no usages  Javier Gallego *
    NewClass.askforNewBounty( onePiecePer: this);
}
```

```
package OnePiece;

import java.util.Scanner;

public class NewClass {  1 usage  new *
    static void askforNewBounty(OnePiecePer onePiecePer) {  1 usage  new *
        long cantidad;
        Scanner teclado =new Scanner (System.in);
        System.out.println("Introduce la nueva recompensa:");
        cantidad=teclado.nextLong();
        onePiecePer.recompensa =cantidad;
    }
}
```

After

Type: Bloaters

DEAD CODE SMELL

Type: Dispensables

```
public OnePiecePer (){  1 usage   Javier Gallego
  this.Nombre="";
  this.recompensa=0;
  this.frutaDiablo="";
  this.tripulacion="";
  this.posicion="";
  this.haki=false;
  this.especie="";

}

public OnePiecePer (String Nombre, long recompensa, String frutaDiablo, String tripulacion, String posicion,
  this.Nombre=Nombre;
  this.recompensa=recompensa;
  this.frutaDiablo=frutaDiablo;
  this.tripulacion=tripulacion;
  this.posicion=posicion;
  this.haki=haki;
  this.especie=especie;

}
```

Before

```
public class OnePiecePer {  14 usages   Javier Gallego *
    public String Nombre;  3 usages
    public long recompensa;  6 usages
    public String frutaDiablo;  4 usages
    public String tripulacion;  4 usages
    public String posicion;  3 usages
    public boolean haki;  5 usages
    public String especie;  4 usages


    public OnePiecePer (String Nombre, long recompensa, String frutaDiablo
      this.Nombre=Nombre;
      this.recompensa=recompensa;
      this.frutaDiablo=frutaDiablo;
      this.tripulacion=tripulacion;
      this.posicion=posicion;
      this.haki=haki;
      this.especie=especie;

    }
```

After

In this step we removed the constructor because it was taking up the entire code

LONG PARAMETER LIST SMELL

Before

```
public OnePiecePer (String Nombre, long recompensa, String
  this.Nombre=Nombre;
  this.recompensa=recompensa;
  this.frutaDiablo=frutaDiablo;
  this.tripulacion=tripulacion;
  this.posicion=posicion;
  this.haki=haki;
  this.especie=especie;
```

Type: Bloaters

After

```
public OnePiecePer (String Nombre, long recompensa, String frutaDiablo, String tripulacion) {
  this.Nombre=Nombre;
  this.recompensa=recompensa;
  this.frutaDiablo=frutaDiablo;
  this.tripulacion=tripulacion;
}
```

In the normal constructor we remove the last 3 variables.

```
OnePiecePer C1 =new OnePiecePer ( Nombre: "Monkey D. Luffy", recompensa: 30000000, frutaDiablo: "Hito Hito no mi Modelo Nika", tripulacion: "Mugiwaras");
OnePiecePer C2 =new OnePiecePer ( Nombre: "Kaido", recompensa: 50000000, frutaDiablo: "Uo Uo no mi Modelo Dragón", tripulacion: "Piratas Bestia");
OnePiecePer C3 =new OnePiecePer ( Nombre: "Buggy", recompensa: 30000000, frutaDiablo: "Bara Bara no mi", tripulacion: "Cross Guild");
OnePiecePer C4 =new OnePiecePer ( Nombre: "Foxy", recompensa: 500000, frutaDiablo: "Noro Noro no mi", tripulacion: "Piratas de Foxy");
```

INAPPROPIATE INTIMACY SMELL

```
public class OnePiecePer {  15 usages  Javier Gallego *
    public String Nombre;  4 usages
    public long recompensa;  7 usages
    public String frutaDiablo;  5 usages
    public String tripulacion;  5 usages
    public String posicion;  2 usages
    public boolean haki;  4 usages
    public String especie;  3 usages
```

Before

In this case we changed the variables from public to private.

Type: Couplers

After

```
public class OnePiecePer {  15 usages  Javier Gallego *
    private String Nombre;  4 usages
    private long recompensa;  6 usages
    private String frutaDiablo;  5 usages
    private String tripulacion;  5 usages
    private String posicion;  2 usages
    private boolean haki;  4 usages
    private String especie;  3 usages
```

DUPLICATE CODE SMELL

```
public String toString()    Javier Gallego
   {
     return "Nombre: " + this.Nombre + "\n" + "Recompensa: " + this.rec
   }

public void incrementarRecompensa(long cantidad) {  no usages  Javier Galleg
    NewClass.askforNewBounty( onePiecePer: this);
}


public void mostrarInformacion(){  no usages  Javier Gallego
    System.out.println(this.toString());
}
```

Before

We removed the mostrarInformacion method, since having toString makes it unnecessary.

Type: Dispensables

After

```
public String toString()    Javier Gallego
   {
     return "Nombre: " + this.Nombre + "\n" + "Recompensa: " + this.recom
   }

public void incrementarRecompensa(long cantidad) {  no usages  Javier Gallego
    NewClass.askforNewBounty( onePiecePer: this);
}
```

LONG METHOD SMELL

```
package OnePiece;

import java.util.Scanner;

public class Main {  no usages  Javier Gallego

    public static void main(String[] args) {  no usages  Javier Gallego
        OnePiecePer[] Personaje=new OnePiecePer[4];

        Scanner teclado =new Scanner (System.in);

        OnePiecePer C1 =new OnePiecePer ( Nombre: "Monkey D. Luffy",  recompensa: 30000000,  frutaDiablo: "Hito Hi
        OnePiecePer C2 =new OnePiecePer ( Nombre: "Kaido",  recompensa: 50000000,  frutaDiablo: "Uo Uo no mi Model
        OnePiecePer C3 =new OnePiecePer ( Nombre: "Buggy",  recompensa: 30000000,  frutaDiablo: "Bara Bara no mi",
        OnePiecePer C4 =new OnePiecePer ( Nombre: "Foxy",  recompensa: 500000,  frutaDiablo: "Noro Noro no mi",
```

Before

We created a method that returns a character with Ctrl + Alt + M

Type: Bloaters

After

This will extract a method from the code we have selected.

```
private static OnePiecePer[] getOnePiecePers() {  1 usage  new *
    OnePiecePer[] Personaje=new OnePiecePer[4];
    return Personaje;
}
```

SWITCH STATEMENTS SMELL

```
public void esPeligroso() {  1 usage   Javier Gallego
    if(recompensa>=100000000 ||haki==true) {
        System.out.println("Es peligroso");
    }
    else
    {
        System.out.println("No es peligroso");
    }
}
```

Before

We take the esPeligroso method and refactor it so that it does not directly print the if statement, but instead prints the getCategory method that has been automatically created.

After

```
public void esPeligroso() {  1 usage   Javier Gallego *
    getCategory( onePiecePer: this);
}

private static void getCategory(OnePiecePer onePiecePer) {  1 usage   Javier Galleg
    if(onePiecePer.recompensa >=100000000 || onePiecePer.haki ==true) {
        System.out.println("Es peligroso");
    }
    else
    {
        System.out.println("No es peligroso");
    }
}
```

Type: OO Abusers

TEMPORARY FIELDS SMELL

```
Scanner teclado =new Scanner (System.in);
```

Before

Type: OO Abusers

We cut the scanner and paste it inside our getOnePiecePer method

After

```
private static OnePiecePer[] getOnePiecePers() {  1 usage   DJGD0 *
    OnePiecePer[] Personaje = new OnePiecePer[4];
    Scanner teclado = new Scanner(System.in);
    return Personaje;
}
```

FEATURE ENVY SMELL

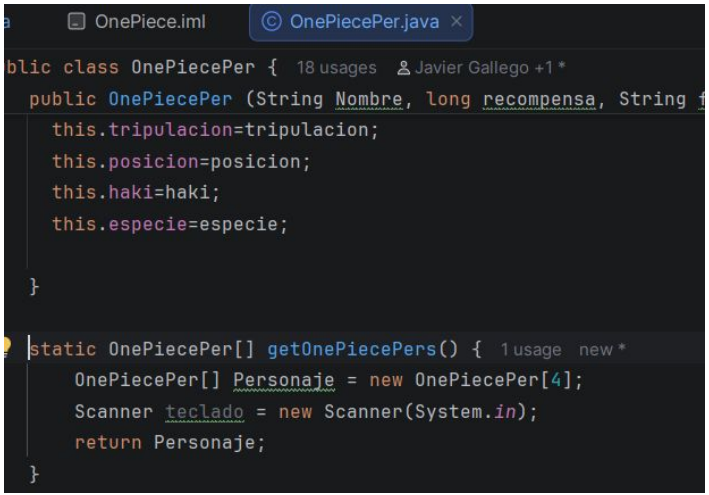
```
private static OnePiecePer[] getOnePiecePers() {  1 usage   DJGD0 *
    OnePiecePer[] Personaje = new OnePiecePer[4];
    Scanner teclado = new Scanner(System.in);
    return Personaje;
}
```

Before

Type: Couplers

We put the OnePiecePers method inside the main method.

After



SHOTGUN SURGERY SMELL

```
", recompensa: 30000000, frutaDiablo: "Hito Hito no mi Modelo Nika", tripulacion: "Mugiwaras");
nsa: 50000000, frutaDiablo: "Uo Uo no mi Modelo Dragón", tripulacion: "Piratas Bestia");
nsa: 30000000, frutaDiablo: "Bara Bara no mi", tripulacion: "Cross Guild");
sa: 500000, frutaDiablo: "Noro Noro no mi", tripulacion: "Piratas de Foxy");
```

Before

Type: Change Preventers

Within the characters shown in the array, I change the quotation marks where it says "Mugiwaras" to the data OnePiecePer.MUGIWARAS.

```
compensa: 30000000, frutaDiablo: "Hito Hito no mi Modelo Nika", OnePiecePer.MUGIWARAS, posicion: "
0000000, frutaDiablo: "Uo Uo no mi Modelo Dragón", tripulacion: "Piratas Bestia", posicion: "Capitán"
```

Next, we created the final static file that prints “Mugiwaras” to the OnePiecePer variables.

After

```
public class OnePiecePer {  19 usages  Javier Gallego +1 *
    public String Nombre;  4 usages
    public long recompensa;  7 usages
    public String frutaDiablo;  5 usages
    public String tripulacion;  5 usages
    public String posicion;  4 usages
    public boolean haki;  6 usages
    public String especie;  5 usages
    public static final String MUGIWARAS = "Mugiwaras";  1 usage
```

MEDAC

Instituto Oficial de Formación Profesional